

Empirical Testing Harness for Multigraph Extension Problem

This folder contains a comprehensive benchmarking suite for evaluating the **Minimal Extension Problem** algorithms on multigraphs. The suite includes tools for synthetic data generation, automated experiment execution, statistical analysis, and publication-quality visualization.

Table of Contents

- 1. [Overview](#)
- 2. [Problem Definition](#)
- 3. [Directory Structure](#)
- 4. [Prerequisites](#)
- 5. [Quick Start](#)
- 6. [Experiment Suite](#)
- 7. [Running Experiments](#)
- 8. [Data Generation](#)
- 9. [Output Format](#)
- 10. [Analysis Tools](#)
- 11. [Reproducibility](#)

Overview

The benchmarking harness evaluates two algorithms for the Minimal Extension Problem:

Algorithm	Type	Complexity	Use Case
Exact	Branch & Bound	Exponential ($O(n! \cdot n^2)$)	Small graphs ($n \leq 10$)
Approx	Hungarian-based Heuristic	Polynomial ($O(n^3)$)	Large graphs ($n \leq 100$)

The suite measures:

- **Solution Quality:** Approximation ratio $\rho = \frac{C_{\text{approx}}}{C_{\text{exact}}}$
- **Runtime Performance:** Wall-clock execution time in milliseconds
- **Scalability:** How runtime grows with graph size
- **Robustness:** Performance across varying graph densities

Problem Definition

Given two multigraphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$ where $|V_G| \leq |V_H|$, the **Minimal Extension Problem** seeks to find the minimum number of edges that must be added to H such that G becomes a subgraph of H .

Formal Definition:
$$\min_{f: V_G \rightarrow V_H} \sum_{(u,v) \in E_G} \max(0, m_G(u,v) - m_H(f(u), f(v)))$$

Where:

- f is an injective mapping from vertices of G to vertices of H
- $m_G(u,v)$ is the edge multiplicity between u and v in G
- The cost represents edges that must be added to H to accommodate G

Directory Structure

```
benchmark/
├── Makefile                # Build and run targets
├── pyproject.toml          # Python dependencies (uv/pip)
├── README.md              # This documentation
├── src/
│   ├── ExperimentRunner.py # Main experiment orchestration script
│   ├── Generator.cpp       # C++ random multigraph generator
│   └── Runner.cpp          # Legacy C++ benchmark runner
├── scripts/
│   └── plot_results.py     # Standalone plotting utility
├── analysis.ipynb         # Interactive Jupyter analysis notebook
├── bin/                   # Compiled binaries (generated)
│   ├── generator
│   └── runner
└── experiments/           # Experiment outputs (generated)
    ├── exp_YYYYMMDD_HHMMSS/
    │   ├── results.csv
    │   └── visualizations/
    │       ├── acc_s*_G.png
    │       ├── acc_s*_H.png
    │       ├── comparison_G.png
    │       ├── comparison_H.png
    │       ├── comparison_Exact.png
    │       └── comparison_Approx.png
```

Prerequisites

1. Build the Implementation

The benchmarking tools require the compiled algorithm binary from the `implementation/` folder:

```
cd ../implementation
make
```

This produces `implementation/bin/main.o` which is invoked by the experiment runner.

2. Python Environment

Dependencies are managed via `uv` (recommended) or standard `pip`.

Option A: Using uv (Recommended)

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Sync dependencies from pyproject.toml
uv sync
```

Option B: Using pip

```
pip install pandas matplotlib seaborn networkx jupyter
```

Dependencies

Package	Version	Purpose
pandas	≥2.0	Data manipulation and CSV I/O
matplotlib	≥3.7	Plotting and visualization
seaborn	≥0.12	Statistical visualization
networkx	≥3.0	Graph generation and analysis
jupyter	≥1.0	Interactive notebook analysis

Quick Start

```
# 1. Build the implementation
cd ../implementation && make && cd ../benchmark

# 2. Sync Python dependencies
uv sync

# 3. Run all experiments
make run-experiments

# 4. Open analysis notebook
jupyter lab analysis.ipynb
```

Experiment Suite

The experiment suite (`src/ExperimentRunner.py`) contains five distinct experiment types:

Experiment 1: Accuracy Analysis

Purpose: Evaluate approximation quality by comparing heuristic solutions against optimal solutions.

Parameter	Value
Graph sizes ($ V_G $)	3, 4, 5, 6, 7
Host graph offset ($ V_H - V_G $)	+2 vertices
Edge density	0.5
Max edge multiplicity	2
Iterations per size	30
Total test cases	150

Metrics Collected:

- `exact_cost`: Optimal extension cost (ground truth)
- `approx_cost`: Heuristic extension cost
- `exact_time`: Exact algorithm runtime (ms)
- `approx_time`: Approx algorithm runtime (ms)
- `exact_found` / `approx_found`: Whether $G \subseteq H$ (cost = 0)

Analysis: Computes the approximation ratio $\rho = C_{\text{approx}} / C_{\text{exact}}$ for cases where $C_{\text{exact}} > 0$. A ratio of 1.0 indicates optimal solution; higher values indicate suboptimality.

Experiment 2: Scalability Analysis

Purpose: Measure runtime growth as graph size increases.

2a. Exact Algorithm Scalability

Parameter	Value
Graph sizes ($ V_G $)	3, 4, 5, 6, 7, 8, 9
Host graph offset	+2 vertices
Edge density	0.4
Max edge multiplicity	2
Iterations per size	5
Total test cases	35

2b. Approximate Algorithm Scalability

Parameter	Value
Graph sizes (\$ V_G \$)	5, 6, 7, 8, 10, 15, 20, 30, 40, 50
Host graph offset	+5 vertices
Edge density	0.4
Max edge multiplicity	2
Iterations per size	5
Total test cases	50

Metrics Collected:

- **algorithm**: "exact" or "approx"
- **size_G**, **size_H**: Vertex counts
- **time**: Execution time (ms)
- **cost**: Extension cost

Analysis: Plots runtime vs. graph size on log scale to visualize exponential (Exact) vs. polynomial (Approx) growth.

Experiment 3: Density Impact Analysis

Purpose: Analyze how edge density affects algorithm performance.

3a. Approximate Algorithm (Medium/Large Graphs)

Parameter	Value
Graph sizes (\$ V_G \$)	20, 25
Host graph offset	+3 vertices
Densities tested	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9
Max edge multiplicity	2
Iterations per (size, density)	5
Total test cases	90

3b. Exact Algorithm (Small Graphs)

Parameter	Value
Graph size (\$ V_G \$)	6
Host graph offset	+2 vertices
Densities tested	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9

Parameter	Value
Max edge multiplicity	2
Iterations per density	30
Total test cases	270

Metrics Collected:

- **density**: Edge probability (0.0 to 1.0)
- **time**: Execution time (ms)
- **cost**: Extension cost

Analysis: Box plots showing runtime distribution across densities. Higher density generally increases runtime due to larger search space.

Experiment 4: Visual Comparison

Purpose: Generate visual representations of algorithm outputs for qualitative analysis.

Parameter	Value
Graph size ($ V_G $)	7
Host graph offset	+2 vertices
Edge density	0.6
Max edge multiplicity	3
Random seeds	G: 42, H: 123

Outputs:

- **comparison_G.png**: Pattern graph G (circular layout)
- **comparison_H.png**: Host graph H (circular layout)
- **comparison_Exact.png**: H with optimal extension edges (red dashed)
- **comparison_Approx.png**: H with heuristic extension edges (red dashed)

Visualization Legend:

- **Light blue/green nodes**: Graph vertices
- **Black edges**: Original edges in H
- **Red dashed edges**: Added extension edges

Experiment 5: Cost vs N (Simultaneous Embeddings)

Purpose: Analyze how the extension cost grows as we demand more simultaneous vertex-disjoint embeddings of G into H . The parameter N controls how many disjoint copies of the pattern graph must be found in the host graph.

Hypothesis: As N increases, the extension cost should grow because:

- 1. Each embedding consumes $|V_G|$ vertices from H, leaving fewer options for subsequent embeddings
- 2. The embeddings must be vertex-disjoint, creating competition for "good" vertex assignments
- 3. Eventually, H runs out of suitable substructures and edges must be added

Algorithm Parameter: The N value is passed as the third argument to the binary:

```
./main.o <algo> <file> <N>
```

5a. Approximate Algorithm (Multiple Densities)

Parameter	Value
Pattern graph size ($ V_G $)	5
Host graph size ($ V_H $)	25
N values tested	1, 2, 3, 4, 5
Densities tested	0.3, 0.5, 0.7
Max edge multiplicity	2
Iterations per (N, density)	10
Total test cases	150

5b. Exact Algorithm (Ground Truth)

Parameter	Value
Pattern graph size ($ V_G $)	4
Host graph size ($ V_H $)	12
N values tested	1, 2, 3
Edge density	0.5
Max edge multiplicity	2
Iterations per N	10
Total test cases	30

Metrics Collected:

- **size_G, size_H:** Vertex counts
- **n_mappings:** Number of required simultaneous embeddings (N)
- **density:** Edge density
- **time:** Execution time (ms)

- **cost**: Extension cost (edges to add)

Analysis:

- Plot cost vs. N for each density
- Expect superlinear or exponential growth as N approaches $\lfloor |V_H| / |V_G| \rfloor$
- Compare Exact vs Approx cost trajectories
- Higher density graphs may show different growth patterns

Key Insight: This experiment quantifies the "embedding saturation" effect - as we demand more simultaneous copies of G in H, the available vertex space shrinks and more edge additions become necessary to satisfy all N embeddings.

Running Experiments

Run All Experiments

```
make run-experiments
```

Or equivalently:

```
uv run src/ExperimentRunner.py
```

Run Specific Experiments

Use the **ARGS** parameter to select specific experiment types:

```
# Run only accuracy tests
make run ARGS="accuracy"

# Run only scalability tests
make run ARGS="scalability"

# Run only density tests
make run ARGS="density"

# Run only visual comparison
make run ARGS="visual"

# Run only cost vs N (embedding deficit) tests
make run ARGS="cost_vs_n"

# Run multiple specific experiments
make run ARGS="accuracy scalability"
```


Direct Python Invocation

```
# Run all
uv run src/ExperimentRunner.py

# Run specific
uv run src/ExperimentRunner.py accuracy density
```

Available Experiment Names

Name	Description	Approx. Runtime
accuracy	Exact vs Approx comparison	~5-10 minutes
scalability	Runtime growth analysis	~3-5 minutes
density	Density impact study	~10-15 minutes
visual	Graph visualizations	~30 seconds
cost_vs_n	Embedding deficit growth	~5-8 minutes

Data Generation

Random Multigraph Generation

The `generate_random_multigraph(n, density, max_multiplicity, seed)` function creates random multigraphs using the following algorithm:

```
for each vertex pair (i, j) where i ≤ j:
    if random() < density:
        multiplicity = random_int(1, max_multiplicity)
        add multiplicity edges between i and j
```

Parameters:

- `n`: Number of vertices
- `density`: Probability of edge existence (0.0 to 1.0)
- `max_multiplicity`: Maximum edges between any vertex pair
- `seed`: Optional random seed for reproducibility

Graph File Format

Graphs are stored in a simple adjacency matrix format:

```
<n_G>
<adj_matrix_G row 1>
```

```
...
<adj_matrix_G row n_G>
<n_H>
<adj_matrix_H row 1>
...
<adj_matrix_H row n_H>
```

Example (G: 3 vertices, H: 4 vertices):

```
3
0 1 2
1 0 1
2 1 0
4
0 1 0 1
1 0 2 0
0 2 0 1
1 0 1 0
```

Output Format

Results CSV Schema

Each experiment run creates a timestamped folder with a **results.csv** file:

Accuracy Experiment Columns

Column	Type	Description
experiment	string	"accuracy"
size_G	int	Vertices in pattern graph
size_H	int	Vertices in host graph
density	float	Edge density used
exact_time	float	Exact algorithm runtime (ms)
approx_time	float	Approx algorithm runtime (ms)
exact_cost	int	Optimal extension cost
approx_cost	int	Heuristic extension cost
exact_found	bool	True if $G \subseteq H$ (cost=0)
approx_found	bool	True if $G \subseteq H$ (cost=0)

Scalability/Density Experiment Columns

Column	Type	Description
experiment	string	"scalability" or "density"
algorithm	string	"exact" or "approx"
size_G	int	Vertices in pattern graph
size_H	int	Vertices in host graph
density	float	Edge density used
time	float	Execution time (ms)
cost	int	Extension cost

Analysis Tools

Jupyter Notebook ([analysis.ipynb](#))

The interactive analysis notebook provides:

- 1. **Automatic Data Loading:** Finds and loads the most recent experiment results
- 2. **Approximation Ratio Analysis:** Box plots showing ρ distribution by graph size
- 3. **Scalability Plots:** Log-scale runtime comparison (Exact vs Approx)
- 4. **Density Impact Visualization:** Runtime distribution across density levels
- 5. **Graph Visualizations:** Inline display of generated graph images

Key Visualizations

Plot	Purpose	Insights
Approximation Ratio Box Plot	Solution quality	Median ρ close to 1.0 indicates good heuristic
Runtime vs Size (Log Scale)	Scalability	Exponential slope for Exact, flat for Approx
Runtime vs Density Box Plot	Robustness	Performance consistency across graph types

Reproducibility

Fixed Seeds for Visual Comparison

The visual comparison experiment uses fixed random seeds:

- Pattern graph G: `seed=42`
- Host graph H: `seed=123`

This ensures identical graphs across runs for consistent visualization.

Experiment Configuration

All experiment parameters are defined in `ExperimentRunner.py`:

```
# Accuracy: 30 iterations, sizes 3-7, density 0.5
# Scalability (Exact): 5 iterations, sizes 3-9, density 0.4
# Scalability (Approx): 5 iterations, sizes 5-50, density 0.4
# Density: 5-30 iterations, 9 density levels (0.1-0.9)
```

Timeout Handling

Each algorithm invocation has a **60-second timeout**. Failed experiments (timeout or error) are logged but excluded from analysis to prevent skewed results.

Makefile Targets

Target	Command	Description
make all	Build C++ tools	Compiles Generator and Runner
make run	Run experiments	Executes ExperimentRunner.py with optional ARGS
make run-experiments	Run all experiments	Executes full experiment suite
make plot	Generate plots	Runs legacy plotting script
make clean	Clean artifacts	Removes binaries, CSVs, and temp files

Statistical Considerations

Sample Sizes

- **Accuracy tests:** 30 iterations per configuration provides sufficient data for statistical significance (Central Limit Theorem)
- **Scalability tests:** 5 iterations balances runtime with measurement reliability
- **Density tests:** 30 iterations for Exact (high variance), 5 for Approx (low variance)

Expected Variance

Experiment	Algorithm	Expected Variance	Reason
Accuracy	Both	Low	Fixed density, similar graph structures
Scalability	Exact	High	Branch & Bound sensitive to graph structure
Scalability	Approx	Low	Deterministic polynomial algorithm
Density	Exact	Very High	Search space varies dramatically with density
Density	Approx	Moderate	More edges = more computation

Approximation Ratio Interpretation

ρ Value	Interpretation
--------------	----------------

ρ Value	Interpretation
1.0	Optimal solution found
1.0 - 1.5	Excellent approximation
1.5 - 2.0	Good approximation
> 2.0	Suboptimal (investigate graph structure)

Troubleshooting

Common Issues

1. "Binary not found" error

```
cd ../implementation && make
```

2. Python import errors

```
uv sync # or pip install -r requirements.txt
```

3. Timeout on large graphs

- Reduce graph size or density
- Exact algorithm times out on $|V| > 9$ typically

4. Empty results CSV

- Check that implementation binary runs correctly
- Verify graph file format is correct